

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
СУМСЬКИЙ ДЕРЖАВНИЙ УНІВЕРСИТЕТ
КАФЕДРА ІНФОРМАЦІЙНИХ ТЕХНОЛОГІЙ

ПОЯСНЮВАЛЬНА ЗАПИСКА

до курсової роботи

з дисципліни «Основи об'єктно-орієнтованого програмування»

Виконав: студент групи КН-51/1
Дудченко А.Є.

Варіант: 3

Перевірила: Ващенко С.М.

ЗМІСТ

ЗМІСТ	2
1 ПОСТАНОВКА ЗАДАЧІ	3
2 ЗАВДАННЯ 1	6
2.1 Опис класу.....	6
2.2 Програмні коди класу.....	9
2.3 Програмні коди модулів виконання завдання.....	14
3 ЗАВДАННЯ 2	20
3.1 Опис класу.....	20
3.2 Програмні коди класу.....	24
3.3 Програмні коди модулів виконання завдання.....	30
4 ГОЛОВНА ПРОГРАМА	35
5 КОНТРОЛЬНИЙ ПРИКЛАД.....	38
СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ	43

1 ПОСТАНОВКА ЗАДАЧІ

Загальна постановка задачі

Розробити програму, яка реалізує роботу з запропонованими класами. Користувач може виконувати одне із практичних завдань (завдання 1 або завдання 2 відповідно). Вибір можливих дій виконується користувачем. Передбачити можливість ознайомлення користувача з завданням роботи.

Всі дії користувача повинні контролюватися. У випадку некоректних дій користувача або виникнення виключної ситуації, передбачити адекватну реакцію, яка має супроводжуватися змістовними коментарями. Під час виконання завдання на екрані має відображатися інформація виключно про поточну роботу, іншої зайвої інформації не має бути.

Кожне практичне завдання передбачає спілкування з користувачем. За результатами виконання будь-якого завдання користувач повинен мати можливість створити звіт у вигляді текстового файлу, який повинен зберігатися на диску.

Введення та/або виведення інформації для класів має виконуватися через роботу стандартних потоків з об'єктами розроблених класів.

Виконання основних дій по реалізації кожного з пунктів меню оформити у вигляді функцій в окремих додаткових файлах (дотримуватися принципу модульності).

Головний файл проєкту повинен містити лише код, який керує процесом роботи програми у відповідності до дій користувача з головним меню.

Завдання 1

Описати клас відповідно таблиці, поданої нижче. Клас повинен бути максимально функціональним та зручним у користуванні.

Для виконання завдання 1 забезпечити користувача можливістю створення масиву із об'єктів описаного класу. Передбачити можливість виведення елементів всього масиву на екран.

Виконати в масиві пошук інформації згідно варіанту. Забезпечити виведення результатів пошуку на екран чи у файл за вибором користувача.

Увага!

Правильно обирайте типи даних для елементів класу. Не нехтуйте основними принципами проєктування!

Враховуйте всі стандартні рекомендації по оформленню коду та наповненню класу.

Також зверніть увагу на вимоги в розділі "Загальна постановка задачі"!!

Таблиця 1.1 – Постановка завдання 1

Клас	Пошук
Клас Music (музичний твір): дата продажу, назва твору, кількість проданих копій, чи продається на DVD.	Визначити загальну кількість проданих копій визначеного користувачем твору.

Завдання 2

Описати клас у відповідності до варіанту з таблиці, що подано нижче. Внести в список атрибутів "колір" (обирається із зазделегідь визначеного переліку). У разі генерації нового об'єкту в методах класу алгоритм формування його кольору визначити самостійно.

Введення та/або виведення об'єктів розробленого класу має виконуватися через роботу цих об'єктів зі стандартними потоками (текстовими та/чи файловими).

Для класу передбачити перевизначення зазначеної у варіанті операції (колонка «Знак операції»). Суть операції та у який спосіб виконати

перевизначення також вказано у таблиці. При реалізації перевизначення враховуйте всі стандартні рекомендації щодо цього.

Перелік методів може бути розширений на власний розсуд студента.

Для виконання завдання за даними, отриманими від користувача, створити необхідну кількість об'єктів розробленого класу та продемонструвати роботу перевизначеної операції.

Увага!

Правильно обирайте типи даних для елементів класу.

Враховуйте всі стандартні рекомендації по оформленню коду та наповненню класу.

Також зверніть увагу на вимоги в розділі "Загальна постановка задачі"!!

Таблиця 1.2 – Постановка завдання 2

Клас	Суть операції	Знак операції	Спосіб перевизначення
Клас описує коло, положення якого задається координатами центру та радіусом.	Операція полягає у формуванні кола, радіус якого дорівнює сумі радіусів кіл-операндів, а центр збігається з центром першого операнда.	%	дружня функція

2 ЗАВДАННЯ 1

2.1 Опис класу

Клас `Music` розроблено для збереження та обробки інформації про музичні записи. Для забезпечення інкапсуляції, всі внутрішні атрибути класу `Music` приховані в закритій секції `private` [3]. Прямий доступ до них ззовні заборонено, а будь-яка робота з об'єктами здійснюється через відкритий інтерфейс `public`, який складається з конструкторів, деструктора, геттерів та сеттерів.

Клас містить три типи конструкторів для гнучкого керування: стандартний (створює безпечний порожній екземпляр за замовчуванням), ініціалізації (заповнює поля реальними даними) та копіювання (забезпечує створення незалежних дублікатів) [1]. Оголошений деструктор автоматично викликається при виході об'єкта з області видимості, виводячи в консоль інформацію про знищення екземпляра для контролю відсутності витоків пам'яті. Операції введення та виведення реалізовані як дружні функції (`friend`), що дозволяє їм мати прямий доступ до приватних полів та інтегрувати об'єкти зі стандартними потоками `std::ostream` та `std::istream` (табл. 2.1) [1].

Таблиця 2.1 – Специфікація членів класу `Music`, рівнів доступу та призначення інтерфейсу

Ідентифікатор	Рівень доступу	Тип даних	Опис призначення
<code>date</code>	<code>private</code>	<code>std::string</code>	Дата продажу музичного твору (формат дд.мм.рррр)

Продовження табл. 2.1

Ідентифікатор	Рівень доступу	Тип даних	Опис призначення
title	private	std::string	Назва музичного твору (ідентифікатор для пошуку)
copies	private	long long	Кількість проданих копій (≥ 0)
onDVD	private	bool	Ознака наявності видання на DVD носії
Music()	public	Конструктор	Стандартний конструктор для створення безпечного пустого об'єкта
Music(std::string date, std::string title, long long copies, bool onDVD)	public	Конструктор	Конструктор ініціалізації об'єкта конкретними даними з валідацією
Music(const Music& other)	public	Конструктор	Конструктор копіювання для створення повноцінного дубліката об'єкта
~Music()	public	Деструктор	Автоматично звільняє ресурси та виводить повідомлення про знищення об'єкта

Продовження табл. 2.1

Ідентифікатор	Рівень доступу	Тип даних	Опис призначення
GetDate() const	public	std::string	Константний геттер, що повертає дату продажу твору
GetTitle() const	public	std::string	Константний геттер, що повертає назву твору
GetCopies() const	public	long long	Константний геттер, що повертає кількість проданих копій
GetOnDVD() const	public	bool	Константний геттер, що повертає DVD-формат
SetDate(std::string date)	public	void	Сеттер для безпечного перезапису дати продажу
SetTitle(std::string title)	public	void	Сеттер для безпечного перезапису назви твору
SetCopies(long long copies)	public	void	Сеттер для перезапису кількості копій з контролем знаку
SetOnDVD(bool onDVD)	public	void	Сеттер для встановлення або зняття ознаки DVD

Продовження табл. 2.1

Ідентифікатор	Рівень доступу	Тип даних	Опис призначення
<code>operator<<</code>	friend	<code>std::ostream&</code>	Дружня операція форматованого виведення об'єкта в текстовий потік
<code>operator>></code>	friend	<code>std::istream&</code>	Дружня операція введення з потоку з валідацією

2.2 Програмні коди класу

Music.h

```
#pragma once
#include <iostream>
#include <string>

class Music {
public:

    Music();
    Music(std::string date, std::string title, long long copies, bool onDVD);
    Music(const Music& other);
    ~Music();
```

```

std::string GetDate()const { return date; };
std::string GetTitle()const { return title; };
long long GetCopies()const { return copies; };
bool GetOnDVD()const { return onDVD; };

```

```

void SetDate(std::string date);
void SetTitle(std::string title);
void SetCopies(long long copies);
void SetOnDVD(bool onDVD);

```

```

friend std::ostream& operator<<(std::ostream& os, const Music& m);
friend std::istream& operator>>(std::istream& is, Music& m);

```

private:

```

std::string date;
std::string title;
long long copies;
bool onDVD;

```

```
};
```

Music.cpp

```

#include "Music.h"
#include <iomanip>

```

```

Music::Music() {
    date = "01.01.2000";

```

```

        title = "Unknown";
        copies = 0;
        onDVD = false;
    }

Music::Music(std::string date, std::string title, long long copies, bool onDVD) {
    this->date = date;
    this->title = title;
    this->copies = copies;
    this->onDVD = onDVD;
}

Music::Music(const Music& other) {
    this->date = other.date;
    this->title = other.title;
    this->copies = other.copies;
    this->onDVD = other.onDVD;
}

Music::~Music() {
    std::cout << "Object " << this->title << " destroyed." << std::endl;
}

void Music::SetDate(std::string date) {
    if (date == "") {
        std::cout << "Error: Date cannot be empty." << std::endl;
        return;
    }
    this->date = date;
}

```

```
}
```

```
void Music::SetTitle(std::string title) {
    if (title == "") {
        std::cout << "Error: Title cannot be empty." << std::endl;
        return;
    }
    this->title = title;
}
```

```
void Music::SetCopies(long long copies) {
    if (copies < 0) {
        std::cout << "Error: Copies cannot be negative." << std::endl;
        return;
    }
    this->copies = copies;
}
```

```
void Music::SetOnDVD(bool onDVD) {
    this->onDVD = onDVD;
}
```

```
std::ostream& operator<<(std::ostream& os, const Music& m) {
    os << std::setw(12) << m.date << " | "
        << std::setw(12) << m.title << " | "
        << std::setw(12) << m.copies << " | ";

    if (m.onDVD) {
        os << "Yes" << std::endl;
    }
}
```

```
    }  
    else {  
        os << "No" << std::endl;  
    }  
    return os;  
}  
  
std::istream& operator>>(std::istream& is, Music& m) {  
    std::cout << "Enter the title (one word, no spaces): ";  
    is >> m.title;  
  
    std::cout << "Enter the sale date: ";  
    is >> m.date;  
  
    std::cout << "Enter the number of copies sold: ";  
    is >> m.copies;  
  
    std::cout << "Is it available on DVD? (1 for Yes, 0 for No): ";  
    is >> m.onDVD;  
  
    return is;  
}
```

2.3 Програмні коди модулів виконання завдання

Task.h

```
#pragma once

void ClearScreen();
void ShowInfo();
void DoTask1();
void PrintHeader();
```

Task.cpp

```
#include "Task.h"
#include "Music.h"
#include <iostream>
#include <fstream>
#include <string>
#include <iomanip>

void ClearScreen() {
    system("cls");
}

void ShowInfo() {
    ClearScreen();
    std::cout << "=== Music Sales Management ===" << std::endl;
    std::cout << "Class: Music" << std::endl;
```

```
std::cout << " - Sale date" << std::endl;
std::cout << " - Title" << std::endl;
std::cout << " - Copies sold" << std::endl;
std::cout << " - Available on DVD (Yes/No)" << std::endl;
std::cout << "Task 1:" << std::endl;
std::cout << "Build an array of Music objects." << std::endl;
std::cout << "Find the total number of sold copies for a title entered by the user."
<< std::endl;

std::cout << "Save the report to a text file." << std::endl;

std::cout << "=====
=====
======" << std::endl;

system("pause");
}

void PrintHeader() {
    std::cout << "-----" <<
std::endl;

    std::cout << std::setw(12) << "Date" << " | "
        << std::setw(12) << "Title" << " | "
        << std::setw(12) << "Copies" << " | "
        << "DVD" << std::endl;

    std::cout << "-----" <<
std::endl;
}

void DoTask1() {
    ClearScreen();
```

```

std::cout << "=== Task 1 ===" << std::endl;

int n;
std::cout << "Enter the number of music records: ";
std::cin >> n;
if (n <= 0) {
    std::cout << "Error: Number must be positive." << std::endl;
    system("pause");
    return;
}

Music* catalog = new Music[n];

for (int i = 0; i < n; ++i) {
    std::cout << "--- Record # " << i + 1 << "---" << std::endl;
    std::cin >> catalog[i];
}

int choice = -1;
while (choice != 0) {
    ClearScreen();
    std::cout << "=== Task 1 Menu ===" << std::endl;
    std::cout << "1. Show catalog" << std::endl;
    std::cout << "2. Search for a title" << std::endl;
    std::cout << "0. Return to main menu" << std::endl;
    std::cout << "===== " << std::endl;
    std::cout << "Your choice: ";
    std::cin >> choice;
}

```

```

if (choice == 1) {
    ClearScreen();
    std::cout << "--- Whole catalog ---" << std::endl;

    PrintHeader();

    for (int i = 0; i < n; i++) {
        std::cout << catalog[i];
    }
    std::cout << "-----" << std::endl;
    system("pause");
}
else if (choice == 2) {
    ClearScreen();
    std::string searchTitle;
    std::cout << "Enter title to search: ";
    std::cin >> searchTitle;

    long long totalCopies = 0;
    bool found = false;

    for (int i = 0; i < n; i++) {
        if (catalog[i].GetTitle() == searchTitle) {
            totalCopies += catalog[i].GetCopies();
            found = true;
        }
    }

    if (found) {

```

```

        std::cout << "Total copies of " << searchTitle << ": " << totalCopies <<
std::endl;
    }
    else {
        std::cout << "Title " << searchTitle << " not found." << std::endl;
    }

    int saveChoice;
    std::cout << "Do you want to save the report in a text file? (1 for Yes, 0 for
No): ";
    std::cin >> saveChoice;

    if (saveChoice == 1) {
        std::ofstream file("report.txt");
        if (file) {
            file << "Report for title: " << searchTitle << std::endl;
            if (found) {
                file << "Total copies of " << searchTitle << ": " << totalCopies <<
std::endl;
            }
            else {
                file << "Title " << searchTitle << " not found." << std::endl;
            }
            file.close();
            std::cout << "Report is saved to 'report.txt'." << std::endl;
        }
        else {
            std::cout << "Error: Could not open the file." << std::endl;
        }
    }

```

```
    }  
    system("pause");  
    }  
}  
delete[] catalog;  
}
```

3 ЗАВДАННЯ 2

3.1 Опис класу

Клас `Circle` описує геометричне коло на площині, стан якого визначається радіусом та координатами центру. Логіка класу повністю ізольована від прямого зовнішнього втручання за допомогою закритої секції `private` [3]. Зміна радіуса та координат здійснюється через сеттери, при цьому метод `setRadius(double)` містить фільтр, який ігнорує встановлення нульових чи від'ємних значень, запобігаючи спотворенню об'єкта.

Згідно з індивідуальним завданням, до списку полів додано атрибут `color`. Випадковий вибір кольору під час генерації нового об'єкта реалізовано у внутрішній функції `generateRandomColor()`. Метод використовує стандартну функцію `rand()`, яка генерує індекс для вибору кольору із фіксованого внутрішнього масиву рядкових констант.

Перевантаження оператора відсотка (%) виконано у вигляді дружньої функції (`friend`) [1]. Такий підхід дозволяє функції безпосередньо працювати з приватними даними двох кл-операндів, повертаючи нове коло, радіус якого рівний сумі радіусів операндів, центр збігається з першим колом, а колір успадковується від більшої за розміром фігури. Дружні оператори потоків введення (`>>`) та виведення (`<<`) забезпечують роботу об'єкта як з консольним інтерфейсом, так і з файловими `std::ofstream` та `std::ifstream` (табл. 3.1).

Таблиця 3.1 – Специфікація членів класу Circle, рівнів доступу та призначення інтерфейсу

Ідентифікатор	Рівень доступу	Тип даних	Опис призначення
x	private	double	Абсциса (X) координати геометричного центру кола
y	private	double	Ордината (Y) координати геометричного центру кола
radius	private	double	Радіус геометричного кола (> 0)
color	private	std::string	Рядкове представлення поточного кольору кола
generateRandom Color()	private	void	Допоміжний метод автоматичного вибору кольору з масиву за допомогою функції rand()
Circle()	public	Конструктор	Стандартний конструктор (центр у точці (0,0), радіус = 1.0, колір генерується випадково)

Продовження табл. 3.1

Ідентифікатор	Рівень доступу	Тип даних	Опис призначення
Circle(double x, double y, double radius)	public	Конструктор	Конструктор ініціалізації кола заданими координатами та радіусом (з перевіркою знаку)
Circle(const Circle& other)	public	Конструктор	Конструктор копіювання для точного дублювання геометричного стану фігури
~Circle()	public	Деструктор	Автоматично звільняє ресурси та виводить повідомлення про знищення об'єкта
getX() const	public	double	Константний геттер для зчитування абсциси центру
getY() const	public	double	Константний геттер для зчитування ординати центру
getRadius() const	public	double	Константний геттер для зчитування поточного радіуса фігури

Продовження табл. 3.1

Ідентифікатор	Рівень доступу	Тип даних	Опис призначення
getColor() const	public	std::string	Константний геттер для зчитування кольору об'єкта
setX(double x)	public	void	Сеттер для зміни координати X
setY(double y)	public	void	Сеттер для зміни координати Y
setRadius(double radius)	public	void	Сеттер для зміни радіуса з обов'язковою геометричною валідацією знаку
setColor(const std::string& color)	public	void	Сеттер для примусової зміни кольору кола з перевіркою на пустий рядок
operator%	friend	Circle	Дружній оператор об'єднання двох кіл за правилом суми радіусів та домінування кольору
operator<<	friend	std::ostream&	Дружня операція форматованого виведення об'єкта в текстовий потік

Продовження табл. 3.1

Ідентифікатор	Рівень доступу	Тип даних	Опис призначення
operator>>	friend	std::istream&	Дружня операція введення з потоку з валідацією

3.2 Програмні коди класу

Circle.h

```
#pragma once
#include <iostream>
#include <string>

class Circle {
public:

    Circle();
    Circle(double x, double y, double radius);
    Circle(const Circle& other);
    ~Circle();

    double getX()const { return x; };
    double getY()const { return y; };
    double getRadius()const { return radius; };
    std::string getColor()const { return color; };
};
```

```

void setX(double x);
void setY(double y);
void setRadius(double radius);
void setColor(const std::string& color);

```

```

friend Circle operator%(const Circle& c1, const Circle& c2);

```

```

friend std::ostream& operator<<(std::ostream& os, const Circle& c);
friend std::istream& operator>>(std::istream& is, Circle& c);

```

```

private:

```

```

    double x;
    double y;
    double radius;
    std::string color;
    void generateRandomColor();

```

```

};

```

Circle.cpp

```

#include "Circle.h"
#include <iomanip>
#include <cstdlib>

```

```

void Circle::generateRandomColor() {

```

```

std::string colors[] = { "Red", "Green", "Blue", "Yellow", "White", "Black"
};

int randomIndex = rand() % 6;
this->color = colors[randomIndex];
}

```

```

Circle::Circle() {
    x = 0.0;
    y = 0.0;
    radius = 1.0;
    generateRandomColor();
}

```

```

Circle::Circle(double x, double y, double radius) {
    this->x = x;
    this->y = y;
    if (radius <= 0) {
        this->radius = 1.0;
    }
    else {
        this->radius = radius;
    }
    generateRandomColor();
}

```

```

Circle::Circle(const Circle& other) {
    this->x = other.x;
    this->y = other.y;
    this->radius = other.radius;
}

```

```

        this->color = other.color;
    }

    Circle::~~Circle() {
        std::cout << "Circle object (Color: " << this->color << ", R: " << this->radius
        << ") destroyed." << std::endl;
    }

    void Circle::setX(double x) {
        this->x = x;
    }

    void Circle::setY(double y) {
        this->y = y;
    }

    void Circle::setRadius(double radius) {
        if (radius <= 0) {
            std::cout << "Error: Radius must be positive." << std::endl;
            return;
        }
        else {
            this->radius = radius;
        }
        generateRandomColor();
    }

    void Circle::setColor(const std::string& color) {
        if (color == "") {

```

```

        std::cout << "Error: Color cannot be empty." << std::endl;
        return;
    }
    else {
        this->color = color;
    }
}

```

```

Circle operator%(const Circle& c1, const Circle& c2) {
    Circle result;
    result.x = c1.x;
    result.y = c1.y;
    result.radius = c1.radius + c2.radius;
    if (c1.radius >= c2.radius) {
        result.color = c1.color;
    }
    else {
        result.color = c2.color;
    }
    return result;
}

```

```

std::ostream& operator<<(std::ostream& os, const Circle& c) {
    os << "Center: (" << c.x << ", " << c.y << ") | "
        << "Radius: " << c.radius << " | "
        << "Color: " << c.color << std::endl;
    return os;
}

```

```

std::istream& operator>>(std::istream& is, Circle& c) {
    std::cout << "Enter X coordinate of the center: ";
    while (!(is >> c.x)) {
        std::cout << "Error: Must be a number. Enter X coordinate: ";
        is.clear();
        is.ignore(10000, '\n');
    }

    std::cout << "Enter Y coordinate of the center: ";
    while (!(is >> c.y)) {
        std::cout << "Error: Must be a number. Enter Y coordinate: ";
        is.clear();
        is.ignore(10000, '\n');
    }

    std::cout << "Enter radius: ";
    double r;
    while (!(is >> r) || r <= 0) {
        std::cout << "Error: Radius must be positive number. Try again: ";
        is.clear();
        is.ignore(10000, '\n');
    }
    c.radius = r;

    c.generateRandomColor();
    return is;
}

```

3.3 Програмні коди модулів виконання завдання

Task2.h

```
#pragma once

void clearScreen2();
void showInfo2();
void doTask2();
```

Task2.cpp

```
#include "Task2.h"
#include "Circle.h"
#include <iostream>
#include <fstream>

void clearScreen2() {
    system("cls");
}

void showInfo2() {
    clearScreen2();
    std::cout << "=== Circle Class ===" << std::endl;
    std::cout << "Class: Circle" << std::endl;
    std::cout << "Attributes:" << std::endl;
    std::cout << " - X coordinate of the center" << std::endl;
    std::cout << " - Y coordinate of the center" << std::endl;
```



```

while (choice != 0) {
    clearScreen2();
    std::cout << "=== Task 2 Menu ===" << std::endl;
    std::cout << "1. Show both circles" << std::endl;
    std::cout << "2. Apply operator% and show the result" << std::endl;
    std::cout << "0. Return to main menu" << std::endl;
    std::cout << "===== " << std::endl;
    std::cout << "Your choice: ";

    while (!(std::cin >> choice)) {
        std::cout << "Error: Invalid input. Try again: ";
        std::cin.clear();
        std::cin.ignore(10000, '\n');
    }

    if (choice == 1) {
        clearScreen2();
        std::cout << "--- Circles Info ---" << std::endl;
        std::cout << "Operand 1: " << c1 << std::endl;
        std::cout << "Operand 2: " << c2 << std::endl;
        std::cout << "-----" << std::endl;
        system("pause");
    }

    else if (choice == 2) {
        clearScreen2();
        Circle result = c1 % c2;
        std::cout << "--- Operation Result % ---" << std::endl;
        std::cout << "Circle 1: " << c1 << std::endl;
        std::cout << "Circle 2: " << c2 << std::endl;
    }
}

```

```

std::cout << "Result: " << result << std::endl;
std::cout << "-----" << std::endl;

int saveChoice;
std::cout << "Do you want to save the report in a text file? (1 for Yes, 0 for
No): ";
while (!(std::cin >> saveChoice) || (saveChoice != 0 && saveChoice != 1))
{
    std::cout << "Error: Enter 1 for Yes or 0 for No: ";
    std::cin.clear();
    std::cin.ignore(10000, '\n');
}

if (saveChoice == 1) {
    std::ofstream file("report2.txt");
    if (file) {
        file << "--- Operation Result % ---" << std::endl;
        file << "Circle 1: " << c1 << std::endl;
        file << "Circle 2: " << c2 << std::endl;
        file << "Result: " << result << std::endl;
        file.close();
        std::cout << "Report is saved to 'report2.txt'." << std::endl;
    }
    else {
        std::cout << "Error: Could not open the file." << std::endl;
    }
}
system("pause");
}

```

}
}

4 ГОЛОВНА ПРОГРАМА

Головний модуль програми керує всією програмою. Структура готового проєкту в розгорнутому вигляді із повним переліком усіх файлів у вікні Solution Explorer представлено на рис. 4.1.

KR1Program.cpp

```
#include "Task.h"
#include "Task2.h"
#include <iostream>
#include <ctime>
#include <cstdlib>

int main() {
    srand(time(NULL));

    int choice = -1;
    while (choice != 0) {
        clearScreen2();
        std::cout << "=== KR1 Program - Music Sales Management ===" <<
std::endl;
        std::cout << "1. Show task info" << std::endl;
        std::cout << "2. Do Task 1 (Music)" << std::endl;
        std::cout << "3. Do Task 2 (Circle)" << std::endl;
        std::cout << "0. Exit" << std::endl;
        std::cout << "===== " << std::endl;
        std::cout << "Your choice: ";
```

```
while (!(std::cin >> choice)) {
    std::cout << "Error: Invalid input. Try again: ";
    std::cin.clear();
    std::cin.ignore(10000, '\n');
}

if (choice == 1) {
    ShowInfo();
    showInfo2();
}
else if (choice == 2) {
    DoTask1();
}
else if (choice == 3) {
    doTask2();
}
else if (choice != 0) {
    std::cout << "Error: Invalid choice. Try again." << std::endl;
    system("pause");
}
}
return 0;
}
```

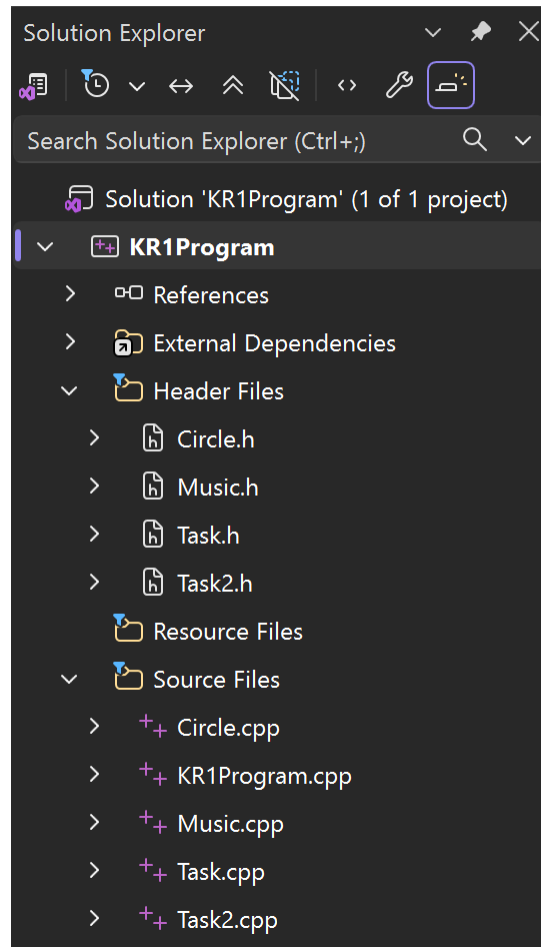


Рисунок 4.1 – Solution explorer

5 КОНТРОЛЬНИЙ ПРИКЛАД

У цьому розділі наведено результати практичного тестування створеного програмного комплексу, продемонстровано навігацію по користувацькому меню, а також відображено реакцію модулів програми на коректність дії користувача та помилки введення числових і символьних даних (рис. 5.1–5.15).

```

=== KR1 Program - Music Sales Management ===
1. Show task info
2. Do Task 1 (Music)
3. Do Task 2 (Circle)
0. Exit
=====
Your choice: A
Error: Invalid input. Try again: 5
Error: Invalid choice. Try again.
Press any key to continue . . .

```

Рисунок 5.1 – Перевірка стійкості головного меню до некоректного введення

```

=== Music Sales Management ===
Class: Music
- Sale date
- Title
- Copies sold
- Available on DVD (Yes/No)
Task 1:
Build an array of Music objects.
Find the total number of sold copies for a title entered by the user.
Save the report to a text file.
=====
Press any key to continue . . .

```

Рисунок 5.2 – Відображення інформаційного вікна для Завдання 1 (Клас Music)

```

=== Circle Class ===
Class: Circle
Attributes:
- X coordinate of the center
- Y coordinate of the center
- Radius
- Color
Available colors: Red, Green, Blue, Yellow, White, Black
Task 2:
Create two Circle objects.
Apply operator% to get a new circle:
- radius = sum of both radii
- center = center of the first circle
- color = color of circle with bigger radius
=====
Press any key to continue . . .

```

Рисунок 5.3 – Відображення інформаційного вікна для Завдання 2 (Клас Circle)

```

=== Task 1 ===
Enter the number of music records: -3
Error: Number must be positive.
Press any key to continue . . .

```

Рисунок 5.4 – Перевірка обробки некоректної кількості записів у Завданні 1

```

=== Task 1 ===
Enter the number of music records: 2
--- Record # 1---
Enter the title (one word, no spaces): Nevermind
Enter the sale date: 24.09.1991
Enter the number of copies sold: 30000000
Is it available on DVD? (1 for Yes, 0 for No): 1
--- Record # 2---
Enter the title (one word, no spaces): Meteora
Enter the sale date: 25.03.2003
Enter the number of copies sold: 27000000
Is it available on DVD? (1 for Yes, 0 for No): 0

```

Рисунок 5.5 – Введення даних для створення каталогу музичних записів

```

=== Task 1 Menu ===
1. Show catalog
2. Search for a title
0. Return to main menu
=====
Your choice: |

```

Рисунок 5.6 – Головне меню підпрограми Задання 1

```

--- Whole catalog ---
-----
      Date |      Title |      Copies | DVD
-----
  24.09.1991 |   Nevermind |   30000000 | Yes
  25.03.2003 |    Meteora |   27000000 | No
-----
Press any key to continue . . .

```

Рисунок 5.7 – Виведення сформованого каталогу музичних платівок у вигляді таблиці

```

Enter title to search: Thriller
Title 'Thriller' not found.
Do you want to save the report in a text file? (1 for Yes, 0 for No): 0
Press any key to continue . . .

```

Рисунок 5.8 – Демонстрація безуспішного пошуку аудіозапису за назвою

```

Enter title to search: Nevermind
Total copies of 'Nevermind': 30000000
Do you want to save the report in a text file? (1 for Yes, 0 for No): 1
Report is saved to 'report.txt'.
Press any key to continue . . .

```

Рисунок 5.9 – Успішний пошук запису, підрахунок кількості копій та збереження звіту у файл

```

=== Task 2 ===
--- Circle 1 ---
Enter X coordinate of the center: A
Error: Must be a number. Enter X coordinate: 5.5
Enter Y coordinate of the center: 4.0
Enter radius: -2
Error: Radius must be positive number. Try again: 3.0
--- Circle 2 ---
Enter X coordinate of the center: 0
Enter Y coordinate of the center: 0
Enter radius: 5.0

```

Рисунок 5.10 – Введення параметрів двох кіл та перевірка реакції програми на помилки користувача

```

--- Circles Info ---
Operand 1: Center: (5.5, 4) | Radius: 3 | Color: Black

Operand 2: Center: (0, 0) | Radius: 5 | Color: Red

-----
Press any key to continue . . .

```

Рисунок 5.11 – Перегляд поточної інформації про створені об'єкти класу Circle

```

--- Operation Result % ---
Circle 1: Center: (5.5, 4) | Radius: 3 | Color: Black
Circle 2: Center: (0, 0) | Radius: 5 | Color: Red
Result: Center: (5.5, 4) | Radius: 8 | Color: Red

-----
Do you want to save the report in a text file? (1 for Yes, 0 for No): 1
Report is saved to 'report2.txt'.
Press any key to continue . . .

```

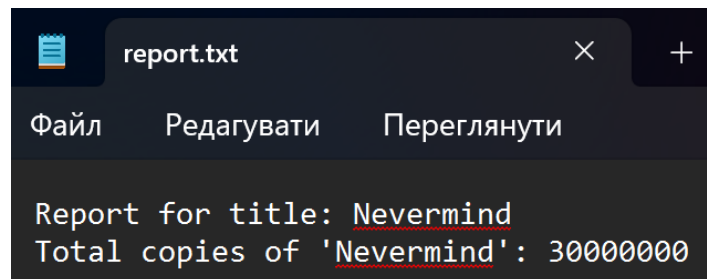
Рисунок 5.12 – Демонстрація роботи перевантаженого оператора % та збереження результатів у текстовий файл

```

=== KR1 Program - Music Sales Management ===
1. Show task info
2. Do Task 1 (Music)
3. Do Task 2 (Circle)
0. Exit
=====
Your choice: 0

```

Рисунок 5.13 – Вихід з програми



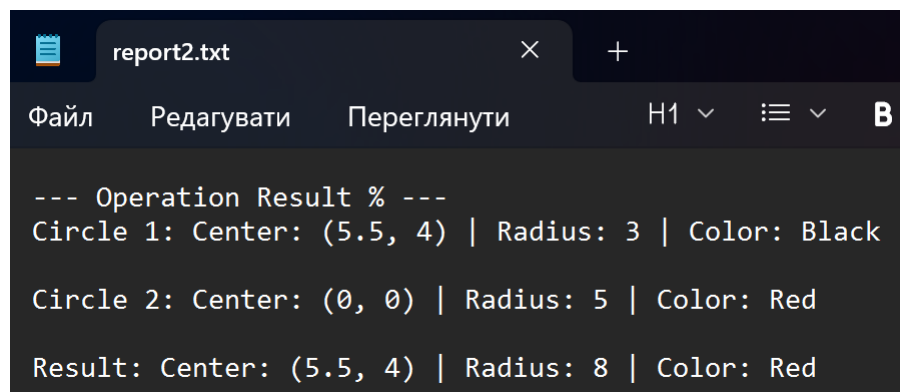
```

report.txt
Файл  Редагувати  Переглянути

Report for title: Nevermind
Total copies of 'Nevermind': 30000000

```

Рисунок 5.14 – Збереження потрібної інформації для Завдання 1 у файл



```

report2.txt
Файл  Редагувати  Переглянути  H1  ≡  B

--- Operation Result % ---
Circle 1: Center: (5.5, 4) | Radius: 3 | Color: Black

Circle 2: Center: (0, 0) | Radius: 5 | Color: Red

Result: Center: (5.5, 4) | Radius: 8 | Color: Red

```

Рисунок 5.15 – Збереження потрібної інформації для Завдання 2 у файл

СПИСОК ВИКОРИСТАНИХ ДЖЕРЕЛ

1. Kirch-Prinz U. A Complete Guide to Programming in C++ [Electronic resource] / U. Kirch-Prinz, P. Prinz. – Mode of access: <https://www.idpoisson.fr/volkov/C++.pdf>. – Title from the screen.
2. W3Schools. C++ Input Validation [Electronic resource]. – Mode of access: https://www.w3schools.com/cpp/cpp_input_validation.asp. – Title from the screen.
3. Освітній портал aCode. Уроки C++ [Електронний ресурс] : курс лекцій з програмування. – Режим доступу: <https://acode.com.ua/uroki-po-cpp/>. – Назва з екрана.